0322. Coin Change

Medium · 1-D Dynamic Programming
leetcode.com/problems/coin-change

Problem

- Minimum number of coins (unlimited supply) to make amount, or -1.

Intuition

- Dp[a] = fewest coins for amount a = 1 + min over coins c of dp[a - c].
- Greedy (largest coin first) fails for coins like [1,3,4] with amount 6 (greedy 4+1+1 vs optimal 3+3).

Brute Force

- Recursion over coin choices: exponential.

Optimal Approach

- Dp[0] = 0, dp[1..amount] = amount + 1 (acts as infinity).
- For a = 1..amount, for c in coins with c <= a: dp[a] = min(dp[a], dp[a-c] + 1).
- Return dp[amount] > amount ? -1 : dp[amount].

Worked Example

```
coins [1,2,5], amount 11.

dp[0..5]  = 0 1 1 2 2 1
dp[6..11] = 2 2 3 3 2 3
answer 3 (5+5+1)
```

Java Solution

```java
public class CoinChange {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;
        for (int a = 1; a <= amount; a++)
            for (int c : coins)
                if (c <= a) dp[a] = Math.min(dp[a], dp[a - c] + 1);
        return dp[amount] > amount ? -1 : dp[amount];
    }
}
```

Complexity

- Time: O(amount coins). Space: O(amount).

Pitfalls

- Using Integer.MAX_VALUE as infinity overflows on + 1; use amount + 1.